

Formatos de secuencias y control de calidad: El formato FASTQ, la escala Phred y el preprocesamiento computacional

BC-CH07 – Biología Computacional

Edición 2 · 2026-09-04

Pregunta del tema. Un secuenciador no entrega letras: entrega letras con una probabilidad de estar equivocadas. ¿Cómo se lee ese número, y qué se hace con las lecturas que no lo pasan?

Producto final. Un módulo propio que lea el formato de lecturas con calidades, convierta los caracteres a puntuaciones y recorte por ventana deslizante, con el criterio de corte declarado.

Mapa del tema

1. El formato FASTQ y la llamada de bases	1
2. La escala de calidad Phred y codificación ASCII	3
3. Diagnóstico con FastQC: Perfiles típicos y anomalías	4
4. Preprocesamiento computacional: Recorte por ventana deslizante	4
5. Genomas de referencia y sistemas de coordenadas	6
6. Diseño computacional en Python: Procesador FASTQ	6
7. Peligros computacionales y actividad de depuración	7
8. Síntesis operativa y tablas de diagnóstico	8
9. Glosario conceptual	8
10. Conexiones y mapa del curso	9
11. Fuentes y reproducibilidad	9

Cómo usar este tema

Este capítulo va del dato crudo al dato utilizable. Empieza por lo que el secuenciador entrega y cómo se codifica la confianza en cada base; sigue por el diagnóstico, que es leer un perfil de calidad y saber qué fenómeno físico lo produce; y termina en el recorte, que es la primera decisión del análisis y ya condiciona todo lo que viene después.

Al terminar sabrá: convertir un carácter de calidad en una probabilidad de error y saber por qué la escala es logarítmica; reconocer los perfiles de calidad típicos y decir qué los causa; explicar por qué se recorta por ventana y no base a base; y no confundir los dos sistemas de coordenadas que conviven en los formatos genómicos.

La ruta **esencial** son la escala de calidad, el formato de lecturas, el diagnóstico de perfiles y el recorte. La ruta de **estudio** añade la codificación antigua, los sistemas de coordenadas y el anexo.

1 El formato FASTQ y la llamada de bases

ESENCIAL Tras el proceso de secuenciación masiva abordado en el capítulo anterior, los detectores ópticos o de corriente iónica generan millones de señales físicas analógicas (trazas de fluorescencia o corrientes eléctricas en picoamperios). El programa de **llamada de bases** convierte esas señales en texto legible, asignando a cada posición una base y una estimación probabilística de la certeza de dicha llamada.

El formato estándar para almacenar y transportar lecturas de secuenciación es el **FASTQ** [3]. Cada lectura ocupa un bloque indivisible de **cuatro líneas**:

1. **Línea 1 (Identificador):** Comienza obligatoriamente con el carácter "@", seguido de un identificador único que codifica los metadatos del instrumento:

```
@A00685:142:H7KNYDSX2:2:1101:1024:1000 1:N:0:ATCACG
```

donde se especifica el ID del secuenciador (A00685), número de corrida (142), código de celda de flujo (H7KNYDSX2), carril (2), tesela óptica (1101), coordenadas X,Y del cúmulo (1024:1000), número de par (1), flag de filtro de calidad (N) e índice de muestra (ATCACG).

2. **Línea 2 (Secuencia):** Cadena de caracteres nucleotídicos brutos en mayúsculas (A, C, G, T), utilizando N para bases no resueltas con ambigüedad.
3. **Línea 3 (Separador):** Comienza obligatoriamente con el carácter "+", pudiendo opcionalmente repetir el identificador de la línea 1.
4. **Línea 4 (Calidades):** Cadena continua de caracteres ASCII que codifica la calidad Phred de cada base, con una longitud exactamente idéntica al número de bases de la línea 2.

El formato FASTQ define un registro estándar de cuatro líneas por lectura: @identificador, secuencia nucleotídica, separador + y cadena de calidad ASCII.

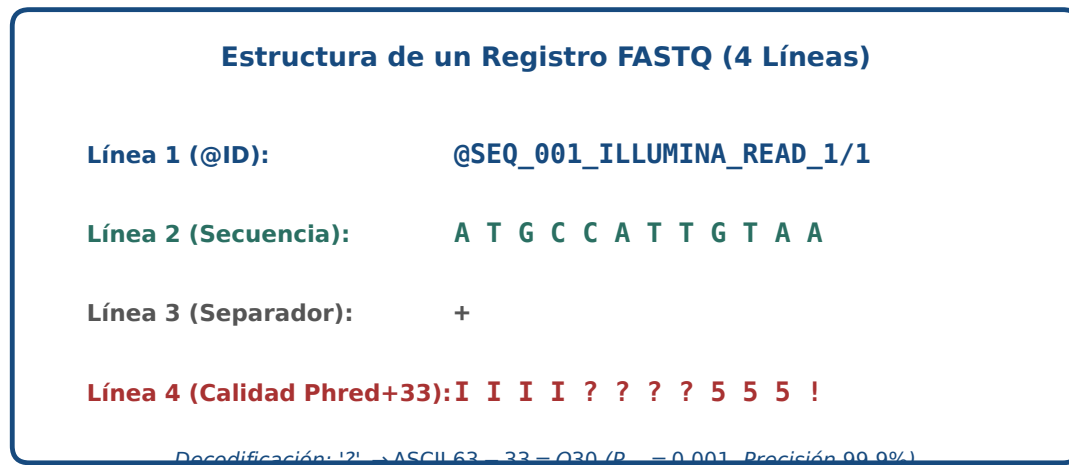


Figura 1: Estructura cuatripartita del formato FASTQ y decodificación de calidades en escala Sanger Phred+33. Figura original generada por `verification/make_figures.py`.

1.1 Lectura simple frente a lectura por pares

- **Lecturas simples (*Single-end*):** El instrumento secuenciar el fragmento de ADN desde un único extremo en dirección 5' → 3', produciendo un único archivo `.fastq` por muestra. Es idóneo para cuantificación de expresión génica en RNA-seq estándar o secuenciación de pequeños ARNs.
- **Lecturas emparejadas (*Paired-end*):** El fragmento biológico (*insert*) se secuenciar desde ambos extremos: la lectura directa hacia adelante (**R1**, `_R1.fastq`) y la lectura inversa complementaria hacia atrás (**R2**, `_R2.fastq`). El conocimiento de la distancia física aproximada entre ambos extremos (*insert size*) proporciona un ancla fundamental para resolver variantes estructurales, reordenamientos cromosómicos y mapeo inequívoco en regiones repetitivas.

La secuenciación single-end produce un archivo FASTQ por muestra mientras que la secuenciación paired-end rinde archivos complementarios directo R1 e inverso R2.

1.2 Traza de análisis sobre la lectura modelo @READ_01

Consideremos el siguiente bloque FASTQ canónico de 8 bases:

```
@READ_01
ATCGATCG
+
IIII????
```

La cadena de calidad "IIII?????" contiene 4 caracteres 'I' (ASCII 73) seguidos de 4 caracteres '?' (ASCII 63). Decodificando con offset +33:

$$\begin{aligned}\text{Calidades } Q &= [40, 40, 40, 40, 30, 30, 30, 30] \\ \text{Media aritmética } \bar{Q} &= \frac{4 \times 40 + 4 \times 30}{8} = \frac{160 + 120}{8} = 35.00 \\ \text{Probabilidad media de error } \bar{P}_{\text{err}} &= \frac{4 \times 10^{-4} + 4 \times 10^{-3}}{8} = 0.000550\end{aligned}$$

`parse_fastq_block` parsea la lectura modelo @READ_01 con calidades IIII???? rindiendo Q medio = 35.00 y probabilidad de error media 0.000550.

Se reproduce con `verification/chapter_checks.py parse_fastq`.

1.3 Qué dice el identificador de una lectura

ESTUDIO La primera línea de cada lectura no es un nombre arbitrario: codifica de dónde salió. En el formato más extendido, los campos van separados por dos puntos y son el identificador del instrumento, el número de la carrera, el identificador de la celda, el carril, el cuadrante, y las coordenadas del grupo dentro de ese cuadrante. Después, tras un espacio, el número de lectura del par, un indicador de filtrado y el código de la muestra.

Concepto. Esa estructura sirve para algo muy concreto: rastrear un problema hasta su origen físico. Si el control de calidad muestra que un subconjunto de lecturas es malo, agrupar por carril suele decir si el problema fue de una zona de la celda o de toda la carrera. Un identificador que se conserva a lo largo del análisis permite hacer esa pregunta; uno que se reenumera al vuelo, no. Por eso conviene no reescribir los identificadores salvo que haya una razón, y documentarla si se hace.

2 La escala de calidad Phred y codificación ASCII

Desarrollada originalmente por Brent Ewing y Phil Green en 1998 para el programa *phred*, la **puntuación de calidad Phred** (Q) cuantifica de manera logarítmica la probabilidad de que una base haya sido llamada incorrectamente por el secuenciador (P_{error}):

$$Q = -10 \log_{10}(P_{\text{error}}) \iff P_{\text{error}} = 10^{-Q/10}$$

La escala de calidad Phred define la probabilidad de error logarítmicamente como $Q = -10 \log_{10}(P_{\text{error}})$ y $P_{\text{error}} = 10^{-Q/10}$.

2.1 Significado biológico y clínico de la escala Phred

Puntuación Phred (Q)	Probabilidad de error (P_{err})	Precisión de la base	Estándar en bioinformática
$Q = 10$	1 en 10 (0.10)	90.0%	Inaceptable (filtrado)
$Q = 20$	1 en 100 (0.01)	99.0%	Mínimo aceptable histórico
$Q = 30$	1 en 1.000 (0.001)	99.9%	Estándar de calidad NGS
$Q = 40$	1 en 10.000 (0.0001)	99.99%	Alta precisión óptima
$Q = 50$	1 en 100.000 (0.00001)	99.999%	Consenso ultra-fiel

2.2 El desplazamiento de 33 y por qué existe

Para guardar cada puntuación, un entero de 0 a 42, en un solo byte de texto imprimible, hay que evitar los códigos por debajo de 33: los caracteres de control, del 0 al 31, y el espacio, el 32, que son imposibles de distinguir a simple vista dentro de una línea. Por eso el estándar suma un desplazamiento constante de 33:

$$Q = \text{ord}(\text{carácter}) - 33 \iff \text{carácter} = \text{chr}(Q + 33)$$

El formato FASTQ Sanger estándar codifica las puntuaciones Phred con un desplazamiento ASCII de 33 mediante $Q = \text{ord}(\text{char}) - 33$.

Traza del carácter de calidad '?':

El signo de interrogación '?' posee el código ASCII decimal 63:

1. $Q = 63 - 33 = 30$
2. $P_{\text{error}} = 10^{-30/10} = 10^{-3} = 0.001000$ (precisión del 99.9%).

El carácter de calidad ? (ASCII 63) evalúa a $Q = 30$ y probabilidad de error $P_{\text{err}} = 0.001000$ (precisión del 99.9 por ciento).

Se reproduce con `verification/chapter_checks.py phred "?"`.

Control defensivo de caracteres fuera de rango:

`decode_phred_string` lanza `ValueError` al encontrar caracteres ASCII inválidos por debajo de 33 como espacios en blanco.

Se reproduce con `verification/chapter_checks.py phred " "`.

3 Diagnóstico con FastQC: Perfiles típicos y anomalías

La herramienta estándar para el control de calidad preliminar en crudo de archivos FASTQ es **FastQC** (Andrews, 2010), cuyos módulos diagnósticos principales revelan la firma biofísica del secuenciador:

1. **Per base sequence quality (Calidad por posición)**: Gráfico de cajas y bigotes a lo largo de las posiciones de la lectura (1 a 150 pb). Muestra típicamente una **degradación progresiva de la calidad hacia el extremo 3'** debido a la pérdida de sincronía de fase en los cúmulos (*phasing* y *pre-phasing*) y al agotamiento progresivo de reactivos enzimáticos.
2. **Per base sequence content (Composición de bases)**: Gráfico del porcentaje de A, C, G y T en cada ciclo. En genomas normales las líneas deben ser paralelas y reflejar el %GC global. Sin embargo, en librerías de RNA-seq preparadas mediante **cebado con hexámeros aleatorios** (*random hexamer priming*), los primeros 10–12 nucleótidos del extremo 5' exhiben fluctuaciones extremas y reproducibles debido a la hibridación no puramente aleatoria de los oligonucleótidos cebadores.
3. **Adapter Content (Contenido de adaptadores)**: Detección de secuencias sintéticas de adaptadores Illumina hacia el extremo 3', producidas cuando el fragmento biológico de ADN es más corto que la longitud de lectura (*adapter read-through*).

Los perfiles diagnósticos de FastQC capturan la degradación progresiva de calidad en 3', los sesgos de cebado por hexámeros aleatorios en 5' y el enriquecimiento de adaptadores.

4 Preprocesamiento computacional: Recorte por ventana deslizante

Para sanear las lecturas antes del mapeo genómico o ensamblaje, se aplican dos operaciones de limpieza esenciales mediante herramientas como *Trimmomatic* o *fastp*:

4.1 Recorte de adaptadores (*Adapter Trimming*)

El recorte de adaptadores elimina ligadores oligonucleotídicos sintéticos de los extremos 3' cuando la longitud de lectura supera el tamaño del inserto de ADNc.

4.2 Recorte por ventana deslizante (*Sliding Window Trimming*)

En lugar de evaluar cada base individualmente de forma aislada (lo que produciría cortes prematuros ante una única base espuria de baja calidad), el algoritmo de **ventana deslizante** evalúa una ventana móvil de tamaño W bases:

1. Desplaza la ventana de W nucleótidos a lo largo de la lectura de $5' \rightarrow 3'$.

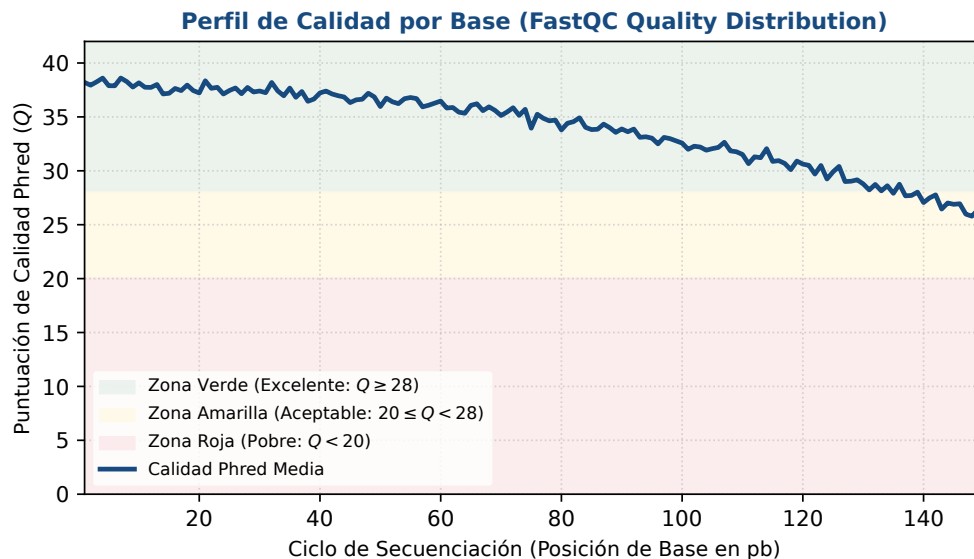


Figura 2: Módulo diagnóstico FastQC de calidad media por posición de base: degradación en extremo 3' y rangos de corte. Figura original generada por `verification/make_figures.py`.

2. Si la calidad media de las W bases cae por debajo de un umbral $Q_{\min}$, la lectura se recorta irrevocablemente en el punto de inicio de la ventana, descartando el resto del extremo 3'.

El recorte por ventana deslizante escanea las lecturas con una ventana móvil de tamaño W y trunca el extremo 3' cuando la calidad media cae por debajo del umbral.

Traza manual para la secuencia modelo ATCGATCGAT:

Sobre una lectura de diez bases cuyas seis primeras calidades son máximas y las cuatro últimas son mínimas ($Q = [40, 40, 40, 40, 40, 40, 0, 0, 0, 0]$), ventana $W = 4$ y umbral $Q_{\min} = 20.0$:

- Ventana 1 (pos 0–3, IIII): Calidades $[40, 40, 40, 40] \Rightarrow \text{Media} = 40.0 \geq 20.0$ (Continúa).
- Ventana 2 (pos 1–4, IIII): Calidades $[40, 40, 40, 40] \Rightarrow \text{Media} = 40.0 \geq 20.0$ (Continúa).
- Ventana 3 (pos 2–5, IIII): Calidades $[40, 40, 40, 40] \Rightarrow \text{Media} = 40.0 \geq 20.0$ (Continúa).
- Ventana 4 (pos 3–6, III!): Calidades $[40, 40, 40, 0] \Rightarrow \text{Media} = 30.0 \geq 20.0$ (Continúa).
- Ventana 5 (pos 4–7, II!!): Calidades $[40, 40, 0, 0] \Rightarrow \text{Media} = 20.0 \geq 20.0$ (Continúa, justo en el umbral).
- Ventana 6 (pos 5–8, I!!!): Calidades $[40, 0, 0, 0] \Rightarrow \text{Media} = \frac{40}{4} = 10.0 < 20.0$ (**Corte en índice 5**).

El fragmento retenido es **ATCGA**, de longitud 5.

Concepto. Observe cómo cae la media: 40, 40, 40, 30, 20 y solo entonces 10, por debajo del umbral. La ventana no mira una base, mira un tramo, y esa es toda la diferencia. En esta lectura, donde la calidad se hunde y ya no se recupera, el criterio base a base cortaría en la posición 6 y la ventana corta en la 5: la ventana se adelanta un poco porque anticipa la caída.

Lo interesante es el caso contrario. Tome una lectura buena con *una* sola base mala en medio, por ejemplo cinco calidades máximas, un cero, y cinco máximas más. El criterio base a base corta en el cero y tira media lectura buena. La ventana de cuatro, en cambio, promedia ese cero con sus vecinos, no baja del umbral en ningún momento y conserva la lectura entera. Para eso sirve la ventana: para no dejar que un error aislado decida por todo el resto.

De ahí que el tamaño de ventana sea un parámetro que hay que declarar junto al resultado. Cambiarlo cambia qué se conserva, y dos análisis con ventanas distintas no son comparables aunque usen el mismo umbral.

`sliding_window_trim` sobre ATCGATCGAT con calidades IIIIIIIII (W=4, $Q_{\min}=20.0$) recorta en la posición

5 produciendo la secuencia ATCGA de longitud 5.

Se reproduce con `verification/chapter_checks.py trim`.

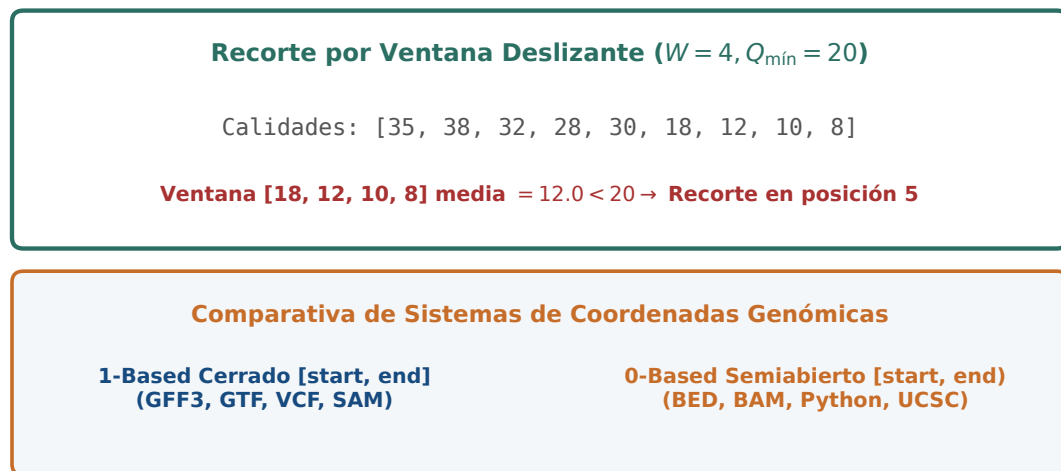


Figura 3: Mecanismo de recorte por ventana deslizante ($W = 4, Q_{\min} = 20$) y convención de sistemas de coordenadas genómicas. Figura original generada por `verification/make_figures.py`.

5 Genomas de referencia y sistemas de coordenadas

El análisis de lecturas procesadas requiere su alineamiento contra un **genoma de referencia**:

- **Formato FASTA multi-cromosoma**: Archivo de texto plano donde cada cromosoma o cóntig comienza con una cabecera `>chrN` seguida de líneas de secuencia nucleotídica continua.
- **Sistemas de coordenadas en genómica**:
 - **1-based cerrado**: Utilizado por formatos orientados a análisis biológico como **SAM**, **VCF** y **GFF**. El primer nucleótido es la posición 1 y un intervalo $[a, b]$ incluye ambos extremos inclusive (longitud $= b - a + 1$).
 - **0-based semiabierto** ($[0, N)$): Utilizado por formatos orientados a computación de rangos rápidos e indexación en memoria como **BED**, **BAM** (**binario interno**) y **arrays de Python**. El primer nucleótido es el índice 0 y el extremo final no está incluido (longitud $= b - a$).

Los genomas de referencia se almacenan en formato FASTA con cabeceras multi-cromosoma y cadenas nucleotídicas continuas.

Los sistemas de coordenadas genómicas distinguen intervalos completamente cerrados 1-based (GFF, VCF, SAM) de intervalos semiabiertos 0-based (BED).

6 Diseño computacional en Python: Procesador FASTQ

A continuación se presenta la implementación modular del procesador FASTQ en Python 3.9:

```
from typing import Dict, List, Tuple

def decode_phred_string(qual_str: str, offset: int = 33) -> List[int]:
    """Decodifica una cadena ASCII de calidades en puntuaciones Phred enteras."""
    scores = []
    for char in qual_str:
        ascii_val = ord(char)
        if ascii_val < offset or ascii_val > offset + 60:
            raise ValueError(f"Caracter de calidad fuera de rango ({ascii_val}) para offset {offset}")
        scores.append(ascii_val - offset)
```

```

return scores

def phred_to_error_prob(q_score: float) -> float:
    """Convierte una puntuacion Phred en probabilidad de error."""
    return round(10.0 ** (-q_score / 10.0), 6)

def parse_fastq_block(block: str, offset: int = 33) -> Dict[str, any]:
    """Parsea un bloque FASTQ de 4 lineas."""
    lines = block.strip().split('\n')
    if len(lines) != 4:
        raise ValueError("El bloque FASTQ debe contener exactamente 4 lineas")
    if not lines[0].startswith('@') or not lines[2].startswith('+'):
        raise ValueError("Formato de cabecera o separador FASTQ invalido")
    seq, qual_str = lines[1].strip(), lines[3].strip()
    if len(seq) != len(qual_str):
        raise ValueError("Longitud de secuencia y calidades no coinciden")
    scores = decode_phred_string(qual_str, offset)
    mean_q = round(sum(scores) / len(scores), 2)
    probs = [10.0 ** (-q / 10.0) for q in scores]
    mean_prob = round(sum(probs) / len(probs), 6)
    return {
        'id': lines[0][1:].strip(),
        'sequence': seq,
        'qualities': scores,
        'mean_q': mean_q,
        'mean_error_prob': mean_prob
    }

def sliding_window_trim(seq: str, qual_str: str, window_size: int = 4, min_q: float = 20.0, offset: int = 33) -> Tuple[str, str]:
    """Recorta el extremo 3' si la calidad media de la ventana cae bajo el umbral."""
    scores = decode_phred_string(qual_str, offset)
    n = len(seq)
    if n < window_size:
        return seq, qual_str
    cut_pos = n
    for i in range(n - window_size + 1):
        window_avg = sum(scores[i:i+window_size]) / window_size
        if window_avg < min_q:
            cut_pos = i
            break
    return seq[:cut_pos], qual_str[:cut_pos]

```

7 Peligros computacionales y actividad de depuración

Omitir la identificación del desplazamiento de calidad (Sanger Phred+33 frente a Illumina 1.3+ Phred+64 histórico) corrompe las puntuaciones de llamada de bases.

Calcular la media aritmética de puntuaciones Phred directamente subestima el error de secuenciación real debido a la transformación logarítmica no lineal.

7.1 Tres errores críticos en preprocesamiento FASTQ

Localice y corrija los tres errores en la siguiente función:

```

def process_fastq_pipeline(fastq_file, is_legacy_illumina):
    # Error 1: Asume siempre offset 33 incluso si los datos provienen de
    # un secuenciador antiguo Illumina 1.3/1.5 (offset 64), interpretando
    # calidades Q=40 como Q=71 o fallando por caracteres invalidos

    # Error 2: Calcula la probabilidad global de error de una lectura haciendo
    # P_error = 10^(-mean(Q)/10) en lugar de la media de las probabilidades individuales
    # (violacion de la desigualdad de Jensen que subestima el error real)

    # Error 3: Al convertir coordenadas BED a intervalos de corte en SAM/VCF,
    # olvida sumar 1 al extremo inicial para transformar 0-based a 1-based

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Desplazamiento erróneo de calidad):** Un fichero con offset +64 decodificado con +33 suma falsamente 31 puntos de calidad a cada base, impidiendo filtrar bases erróneas.
2. **Fallo 2 (Promediación logarítmica incorrecta):** Si una lectura de 2 bases tiene $Q = [40, 10]$, la media de Q es 25 $\Rightarrow P = 10^{-2.5} \approx 0.00316$. Sin embargo, el error real medio es $\frac{0.0001+0.10}{2} = 0.05005 \Rightarrow Q_{\text{real}} \approx 13.0$. La media aritmética de Q infravalora el error en más de un orden de magnitud.
3. **Fallo 3 (Desfase de coordenadas inter-formato):** La posición genómica 100 en BED (0-based) corresponde a la posición 101 en VCF/SAM (1-based).

8 Síntesis operativa y tablas de diagnóstico

8.1 Tabla comparativa de offsets de calidad en FASTQ

Estándar FASTQ	Rango Phred	Desplazamiento	Rango de caracteres ASCII
Sanger / Illumina 1.8+	0 a 41	+33	! a J (ASCII 33–74)
Solexa / Illumina 1.0	–5 a 40	+64	; a h (ASCII 59–104)
Illumina 1.3 / 1.5	0 a 40	+64	@ a h (ASCII 64–104)

8.2 Tabla de diagnóstico de problemas en FastQC

Alerta en FastQC	Causa probable	Comprobación y corrección
Caída brusca de Q en extremo 3'	Desincronización óptica en cúmulos	Aplicar recorte por ventana deslizante (Trimmomatic)
Fluctuación de bases en ciclos 1–12	Cebado por hexámeros aleatorios en RNA-seq	Esperado en RNA-seq; no recortar salvo que afecte mapeo
La señal de adaptador crece hacia el final	el fragmento es más corto que la lectura	recortar adaptadores antes de alinear
Pico bimodal en distribución GC	Contaminación bacteriana o sobreexpresión	Analizar organismos contaminantes mediante FastQ Screen

9 Glosario conceptual

- **FASTQ:** Registro de 4 líneas con secuencia y calidades ASCII.
- **Puntuación de calidad:** escala logarítmica, $Q = -10 \log_{10}(P)$, donde P es la probabilidad de que la base esté equivocada.
- **Phred+33:** Desplazamiento estándar Sanger para calidades legibles.
- **FastQC:** Diagnóstico de calidad y anomalías en lecturas crudas.
- **Ventana deslizante:** Recorte en 3' evaluando medias de calidad.
- **Adaptador:** Secuencia sintética para unión a celda de flujo.
- **Paired-end:** Secuenciación bidireccional en dos archivos (R1/R2).
- **1-based cerrado:** Sistema de coordenadas en SAM, VCF y GFF.
- **0-based semiabierto:** Sistema de indexación en BED y Python.
- **Jensen (desigualdad):** Sesgo al promediar puntuaciones Phred.

10 Conexiones y mapa del curso

Este capítulo completa el preprocesamiento de secuencias en crudo, preparando las lecturas saneadas para el mapeo genómico y alineamiento en BC-CH08.

11 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre formatos de la asignatura, escrita por Álvaro Serrano Navarro. La escala de calidad y su definición logarítmica siguen a Ewing y Green [4]; la especificación del formato de lecturas con calidades y sus codificaciones, a Cock y colaboradores [3]; los módulos de diagnóstico y la lectura de sus perfiles, a Andrews [1]; y el recorte por ventana deslizante, a Bolger y colaboradores [2].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Formato FASTQ, decodificación Phred y filtrado por ventana deslizante

Pregunta, producto y separación de materiales

¿Puede leer el formato de lecturas con calidades, convertir sus caracteres a puntuaciones y decidir dónde recortar, declarando el criterio? Este laboratorio responde que sí.

El producto individual es `mi_control_calidad.py`, con tres funciones: convertir un carácter en puntuación, calcular la media de una cadena de calidades, y recortar por ventana deslizante devolviendo la posición de corte. Lo que se evalúa es que el recorte respete la propiedad que lo justifica: la ventana amortigua, y por eso corta más tarde que un criterio base a base.

El anexo es autónomo: solo necesita la biblioteca estándar. El comprobador prueba su módulo con cadenas de calidad que el enunciado no muestra.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Phred, FASTQ, Trimming)	Decodificación Q=30, parsing FASTQ y corte
3 · Perturbación a Q=20	Recálculo de probabilidad de error ($P = 0.01$)
4 · Guarda de dominio	Captura de <code>ValueError</code> ante <code>ASCII < 33</code>
5 · Preguntas de control	Preguntas sobre offsets y promediado de Q
6 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Preparar el espacio de trabajo

```
python3 --version
./practice_assets/create_workspace.sh BC07_entrega
cd BC07_entrega
```

2 · Escribir el módulo de control de calidad

Tarea. Implemente las tres funciones de `mi_control_calidad.py`: convertir un carácter en puntuación, promediar una cadena de calidades y recortar por ventana deslizante devolviendo la posición de corte. El contrato está en la documentación de la plantilla.

Comprueba. Antes de programar el recorte, resuelva sobre el papel este caso: cinco calidades máximas, una mínima, y cinco máximas más. ¿Dónde corta un criterio base a base? ¿Y una ventana de cuatro con umbral 20? Las dos respuestas son distintas, y la diferencia entre ellas es toda la justificación de recortar por ventana. El comprobador verifica exactamente ese caso.

```
bash herramienta/check_calidad.sh
```

3 • Perturbación a calidad $Q=20$

Evalúe la decodificación del carácter `5' (ASCII 53):

```
python3 herramienta/chapter_checks.py trim > resultados/recorte.txt
cat resultados/recorte.txt
```

La perturbación con el carácter 5 evalúa a una puntuación Phred $Q = 20$ y una probabilidad de error de 0.010000.

Se reproduce con `verification/chapter_checks.py trim`.

4 • Guarda de dominio y control de excepciones

Compruebe que el decodificador rechaza caracteres fuera del rango imprimible estándar (< 33):

```
python3 herramienta/chapter_checks.py phred " " > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza caracteres de calidad fuera de rango lanzando un `ValueError` que identifica el carácter inválido.

Se reproduce con `verification/chapter_checks.py phred " "`.

5 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC07_entrega.tar.gz BC07_entrega
sha256sum BC07_entrega.tar.gz
./practice_assets/check_entrega.sh BC07_entrega BC07_entrega.tar.gz
```

El verificador prueba su módulo con cadenas de calidad que no ha visto, comprueba que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre el efecto del tamaño de ventana, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del parser FASTQ de cuatro líneas y cálculo exacto de la calidad media $\bar{Q} = 35.00$ (Bloque 2).
- **2.0 puntos (20%):** Algoritmo de recorte por ventana deslizante ($W = 4, Q_{\min} = 20.0$) y control de excepciones de la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre el significado de la escala Phred, el impacto de promediar directamente puntuaciones Q y la caída de calidad en 3'.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El carácter `?' decodificó a $Q = 30$.
- La calidad media de la lectura fue $\bar{Q} = 35.00$.
- La secuencia recortada fue ATCGA (5 pb).
- El carácter `5' decodificó a $Q = 20$ ($P = 0.01$).
- Verifiqué que el espacio en blanco lanza ValueError.
- Comprobé que el archivo FASTQ tiene 4 líneas por registro.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, explique por qué promediar aritméticamente puntuaciones Phred subestima la tasa de error global de una lectura frente al promedio de probabilidades reales $10^{-Q/10}$, y describa el mecanismo físico y químico responsable de la pérdida de calidad hacia el extremo 3' en secuenciación Illumina.

Fuentes y reproducibilidad

Este anexo no usa datos externos ni paquetes de terceros. La escala de calidad sigue a Ewing y Green [4], el formato a Cock y colaboradores [3], y el recorte por ventana a Bolger y colaboradores [2].

Bibliografía

- [1] Simon Andrews. Fastqc: A quality control tool for high throughput sequence data. *Babraham Bioinformatics*, 2010. Herramienta estandar de control de calidad para datos de secuenciacion masiva.
- [2] Anthony M. Bolger, Marc Lohse, and Bjoern Usadel. Trimmomatic: A flexible trimmer for illumina sequence data. *Bioinformatics*, 30(15):2114–2120, 2014. Algoritmo canonico de recorte por ventana deslizando y eliminacion de adaptadores.
- [3] Peter J. A. Cock, Christopher J. Fields, Naohisa Goto, Michael L. Heuer, and Peter M. Rice. The sanger fastq file format for sequences with quality scores, and the solexa/illumina fastq variants. *Nucleic Acids Research*, 38(6):1767–1771, 2010. Especificacion canonica del formato FASTQ estandar Sanger Phred+33.
- [4] Brent Ewing and Phil Green. Base-calling of automated sequencer traces using phred. ii. error probabilities. *Genome Research*, 8(3):186–194, 1998. Definicion logaritmica original de la escala de calidad Phred (Q-score).